

Doc. no.:	P0849R6
Date:	2020-12-14
Audience:	EWG
Reply-to:	Zhihao Yuan <zy at miator dot net>

auto(x): *decay-copy* in the language

Changes Since R5

- discuss a few points came across during EWG review
- add a table to summarize the library wording

Changes Since R4

- record LWG feedback
- rebase and refine the wording
- demo examples

Changes Since R3

- complete library wording

Changes Since R2

- dropped `decltype(auto)(x)` in comply with EWG's opinion

Changes Since R1

- propose `decltype(auto)(x)` as well

Changes Since R0

- updated examples

- discussed `decltype(auto)(x)`
- added library wording

Introduction

This paper proposes `auto(x)` and `auto{x}` for casting `x` into a prvalue as if passing `x` as a function argument by value. The functionality appears as the *decay-copy* function in the standard for exposition only.

Motivation

Obtaining a prvalue copy is necessary

A generic way to obtain a copy of an object in C++ is `auto a = x;` but such a copy is an lvalue. We could often convey the purpose in code more accurately if we can obtain the copy as a prvalue. In the following example, let `Container` be a concept,

```
void pop_front_alike(Container auto& x) {  
    std::erase(x.begin(), x.end(), auto(x.front()));  
}
```

If we write

```
void pop_front_alike(Container auto& x) {  
    auto a = x.front();  
    std::erase(x.begin(), x.end(), a);  
}
```

, questions arise – why this is not equivalent to

```
void pop_front_alike(Container auto& x) {  
    std::erase(x.begin(), x.end(), x.front());  
}
```

The problem is, the statement to obtain an lvalue copy is a declaration:

```
auto a = x.front();
```

The declaration's primary purpose is to declare a variable, while the variable being a copy is the declaration's property. In contrast, the expression to obtain an rvalue copy is a clear command to perform a copy:

```
auto(x.front())
```

One might argue that the above is indifferent from

```
T(x.front())
```

However, there are plenty of situations that the `T` is nontrivial to get. We probably don't want to write the original example as

```
void pop_front_alike(Container auto& x) {  
    using T = std::decay_t<decltype(x.front())>;  
    std::erase(x.begin(), x.end(), T(x.front()));  
}
```

Obtaining a prvalue copy with `auto(x)` works always

In standard library specification, we use the following exposition only function to fulfill `auto(x)`'s role:

```
template<class T>  
constexpr decay_t<T> decay_copy(T&& v) noexcept(  
    is_nothrow_convertible_v<T, decay_t<T>>) {  
    return std::forward<T>(v);  
}
```

This definition involves templates, dependent `constexpr`, forwarding reference, `noexcept`, and two traits, and still has caveats if people want to use it in practice. An obvious issue is that `decay_copy(x.front())` copies `x.front()` even if `x.front()` is a prvalue, in other words, a copy.

There is a less obvious issue that needs a code snippet to reproduce:

```

class A {
    int x;

public:
    A();

    auto run() {
        f(A(*this));           // ok
        f(auto(*this));        // ok as proposed
        f(decay_copy(*this));  // ill-formed
    }

protected:
    A(const A&);
};

```

The problem is that `decay_copy` is nobody's friend . We can use `A` directly in this specific example. However, in a more general setting, where a type has access to a set of type `T` 's private or protected copy/move constructors, *decay-copy* an object of `T` fails inside that type's class scope, but `auto(x)` continues to work.

Discussion

auto(x) is a missing piece

Replacing the `char` in `char('a')` with `auto` , we obtain `auto('a')` , which is a function-style cast. Such a formula also supports *injected-class-names* and class template argument deduction in C++17. Introducing `auto(x)` and `auto{x}` significantly improves the language consistency:

variable definition	function-style cast	new expression
<code>auto v(x);</code>	<code>auto(x)</code>	<code>new auto(x)</code>
<code>auto v{x};</code>	<code>auto{x}</code>	<code>new auto{x}</code>
<code>ClassTemplate v(x);</code>	<code>ClassTemplate(x)</code>	<code>new ClassTemplate(x)</code>
<code>ClassTemplate v{x};</code>	<code>ClassTemplate{x}</code>	<code>new ClassTemplate{x}</code>

****** *The type of `x` is a specialization of `ClassTemplate` .*

With this proposal, all the cells in the table copy construct form `x` (due to CTAD's default behavior) to obtain lvalues, prvalues, and pointers to objects, categorized by their columns. Defining `auto(x)` as a library^[1] facility loses orthogonality.

Introducing `auto(x)` into the language even improves the library consistency:

type function style	expression style
<code>void_t<decltype(expr)></code>	<code>decltype(void(expr))</code>
<code>decay_t<decltype(expr)></code>	<code>decltype(auto(expr))</code>

Do we also miss `decltype(auto){x}` ?

`decltype(auto){arg}` can forward `arg` without computing `arg`'s type. It is equivalent to `static_cast<decltype(arg)>(arg)`. If `arg` is a variable of type `T&&`, `arg` is an lvalue but `static_cast<T&&>(arg)` is an xvalue.

EWG discussed this idea, disliked its expert-friendly nature, and concluded that adding this facility would cause the teaching effort to add up.

Does `auto` works in place of *decay-copy* in the library specification?

Not as a simple find-and-replace, but can be made to improve the quality of the library specification.

The background is that, despite being exposition-only, *decay-copy* always materializes its argument and produces a copy. `auto(expr)` is a no-op if the `expr` is a prvalue.

In the library specification where uses *decay-copy*, some do not mean to materialize the expressions; some want a new copy; some do not care. However, with `auto(x)` semantics, we should distinguish the different needs and explicitly say so when a copy is needed.

May a different spelling work better, such as `prvalue_cast` ?

First, `auto(x)` casts to prvalue, but may not be how the name `prvalue_cast` suggests. If there is a `prvalue_cast` keyword, you may expect the following code to pass an array prvalue to the function `foo` :

```
double v[] = { 1.2, 3.5, .8 };
foo(prvalue_cast(v));
```

But

```
foo(auto(v));
```

will pass a `double*`. Therefore, we are talking about a different facility.

On the other hand, the term “prvalue” is unfamiliar to ordinary C++ users. While some people may expect a prvalue-casting without decaying, some other people may not have any expectation when seeing such a term.

In contrast, the use of the `auto` keyword implies that this expression decays. It decays in the same way

```
auto p = v;
```

does and in the same way

```
void bar(auto v);  
bar(v);
```

does. This consistency makes the proposed spelling a lot more teachable comparing to the others.

Can we build a `decay_copy` with a metaprogram that forwards prvalue?

P2237 (<https://wg21.link/p2237>) Metaprogramming means to replace macros, and the answer is yes. Barry kindly provides the following snippet:

```
constexpr void prvalue_cast(meta::info expr) {  
    -> decay_t<|type_of(expr)|>(|expr|);  
}
```

However, we should note that we added `auto` type-specifier in declarations and `new` expressions not because we cannot meta-program in those contexts. We added them because they are useful, so does `auto` in a function-style cast as shown in this paper.

As a side note, it may be a fun thing to see if someone comes up with the following code in the future:

```
auto p = new auto(prvalue_cast(v));
```

Should we also make `static_cast<auto>(x)` work?

The motivation is that because `T(x)` is indifferent from `(T)x` – a C-style cast that may try every possible route to `T`, some codebases may ban `auto(x)` altogether with `T(x)` even though `auto` is a type-placeholder rather than a type. `static_cast<auto>(x)` can be a way to evacuate from such a check.

In terms of styles, without loss of generality, `char{x}` means *direct-list-initialization* from `x`. It allows no narrowing and is not a cast. I believe that no codebase would ban `T{x}`, so `auto{x}` can be an alternative spelling if `auto(x)` is accidentally banned.

If we ignore the motivation for a moment, I think:

1. Whether `static_cast<auto>(x)` should work should also take `static_cast<template-id>(x)` (CTAD) into account. The latter is currently forbidden.
2. If we have such a paper, we may also want to decide whether `static_cast<auto&&>(x)` should work. This expression is a `std::forward` without a need to supply the type, but EWG has already turned down a similar idea (`decltype(auto){x}`).

Demo

Prevent algorithm from modifying through aliases: <https://godbolt.miator.net/z/hhcvbc>

(<https://godbolt.miator.net/z/hhcvbc>)

Using `auto(x)` in rvalue fluent interface: <https://godbolt.miator.net/z/TY8sxr>

(<https://godbolt.miator.net/z/TY8sxr>)

How `auto(x)` assists in defining concepts: <https://godbolt.miator.net/z/GTaaE>

(<https://godbolt.miator.net/z/GTaaE>)

Compare diagnosis to `new auto(x)`: <https://godbolt.miator.net/z/Ks43an>

(<https://godbolt.miator.net/z/Ks43an>)

Wording

The wording is relative to N4868.

Part 1

Modify 7.6.1.4 [expr.type.conv]/1 as indicated:

A *simple-type-specifier* (9.2.9.3) or *typename-specifier* (13.8) followed by a parenthesized optional *expression-list* or by a *braced-init-list* (the initializer) constructs a value of the specified type given the initializer. If the type is a placeholder for a deduced class type, it is replaced by the return type of the function selected by overload resolution for class template deduction (12.4.2.9) for the remainder of this section. Otherwise, if the type is `auto`, it is replaced by the type deduced for the variable `x` in the invented declaration ([dcl.spec.auto] <http://eel.is/c++draft/dcl.spec.auto>):

```
auto x init;
```

, where `init` is the initializer.

Modify 9.2.9.6 [dcl.spec.auto]/5 as indicated:

A placeholder type can also be used in the *type-specifier-seq* in the *new-type-id* or *type-id* of a *new-expression* (7.6.2.8) and as a *decl-specifier* of the *parameter-declaration's decl-specifier-seq* in a *template-parameter* (13.2). The `auto` *type-specifier* can also be used as the *simple-type-specifier* in an explicit type conversion (functional notation). ([expr.type.conv] <http://eel.is/c++draft/expr.type.conv>).

Part 2

[Drafting note: Here is a summary of patterns in the library changes:

Description	Before	Proposed	Alternative
<i>decay-copy</i> specific expression	<i>decay-copy</i> (begin(t))	auto(begin(t))	-
<i>decay-copy</i> unspecific expression	<i>decay-copy</i> (E)	unchanged	auto(identity()(E))
mention of “decayed type”	decayed type of E	type of auto(E)	decay_t<decltype(E)>
explain aftermath of evaluating <i>decay-copy</i>	calls to <i>decay-copy</i> being evaluated in the constructing thread	values produced by auto being materialized in the constructing thread	-

There are three uses of *decay-copy* that this wording does not propose to change. They are in [range.all.general], [range.take], and [range.drop]. –end note]

Modify 24.3.2 [range.access.begin]/2 as indicated:

Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:

- If `E` is an rvalue and `enable_borrowed_range<remove_cv_t<T>>` is `false`, `ranges::begin(E)` is ill-formed.
- Otherwise, if `T` is an array type (6.8.3) and `remove_all_extents_t<T>` is an incomplete type, `ranges::begin(E)` is ill-formed with no diagnostic required.
- Otherwise, if `T` is an array type, `ranges::begin(E)` is expression-equivalent to `t + 0`.
- Otherwise, if `decay-copy-auto (t.begin())` is a valid expression whose type models `input_or_output_iterator`, `ranges::begin(E)` is expression-equivalent to `decay-copy-auto (t.begin())`.
- Otherwise, if `T` is a class or enumeration type and `decay-copy-auto (begin(t))` is a valid expression whose type models `input_or_output_iterator` with overload resolution performed in a context in which unqualified lookup for `begin` finds only the declarations

```
void begin(auto&) = delete;
void begin(const auto&) = delete;
```

then `ranges::begin(E)` is expression-equivalent to `decay-copy-auto (begin(t))` with overload resolution performed in the above context.
- Otherwise, `ranges::begin(E)` is ill-formed.

Modify 24.3.3 [range.access.end]/2 as indicated:

Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:

- If `E` is an rvalue and `enable_borrowed_range<remove_cv_t<T>>` is `false`, `ranges::end(E)` is ill-formed.
- Otherwise, if `T` is an array type (6.8.3) and `remove_all_extents_t<T>` is an incomplete type, `ranges::end(E)` is ill-formed with no diagnostic required.
- Otherwise, if `T` is an array of unknown bound, `ranges::end(E)` is ill-formed.
- Otherwise, if `T` is an array, `ranges::end(E)` is expression-equivalent to `t + extent_v<T>`.
- Otherwise, if ~~`decay-copy-auto`~~ `(t.end())` is a valid expression whose type models `sentinel_for<iterator_t<T>>` then `ranges::end(E)` is expression-equivalent to ~~`decay-copy-auto`~~ `(t.end())`.
- Otherwise, if `T` is a class or enumeration type and ~~`decay-copy-auto`~~ `(end(t))` is a valid expression whose type models `sentinel_for<iterator_t<T>>` with overload resolution performed in a context in which unqualified lookup for `end` finds only the declarations

```
void end(auto&) = delete;
void end(const auto&) = delete;
```

then `ranges::end(E)` is expression-equivalent to ~~`decay-copy-auto`~~ `(end(t))` with overload resolution performed in the above context.
- Otherwise, `ranges::end(E)` is ill-formed.

Modify 24.3.6 [range.access.rbegin]/2 as indicated:

Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:

- If `E` is an rvalue and `enable_borrowed_range<remove_cv_t<T>>` is `false`, `ranges::rbegin(E)` is ill-formed.
- Otherwise, if `T` is an array type (6.8.3) and `remove_all_extents_t<T>` is an incomplete type, `ranges::rbegin(E)` is ill-formed with no diagnostic required.
- Otherwise, if ~~`decay-copy-auto`~~ (`t.rbegin()`) is a valid expression whose type models `input_or_output_iterator`, `ranges::rbegin(E)` is expression-equivalent to ~~`decay-copy-auto`~~ (`t.rbegin()`).
- Otherwise, if `T` is a class or enumeration type and ~~`decay-copy-auto`~~ (`rbegin(t)`) is a valid expression whose type models `input_or_output_iterator` with overload resolution performed in a context in which unqualified lookup for `rbegin` finds only the declarations

```
void rbegin(auto&) = delete;
void rbegin(const auto&) = delete;
```

then `ranges::rbegin(E)` is expression-equivalent to ~~`decay-copy-auto`~~ (`rbegin(t)`) with overload resolution performed in the above context.
- [...]

Modify 24.3.7 [range.access.rend]/2 as indicated:

Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:

- If `E` is an rvalue and `enable_borrowed_range<remove_cv_t<T>>` is `false`, `ranges::rend(E)` is ill-formed.
- Otherwise, if `T` is an array type (6.8.3) and `remove_all_extents_t<T>` is an incomplete type, `ranges::rend(E)` is ill-formed with no diagnostic required.
- Otherwise, if ~~`decay-copy-auto`~~ (`t.rend()`) is a valid expression whose type models `sentinel_for<decltype(ranges::rbegin(E))>` then `ranges::rend(E)` is expression-equivalent to ~~`decay-copy-auto`~~ (`t.rend()`).
- Otherwise, if `T` is a class or enumeration type and ~~`decay-copy-auto`~~ (`rend(t)`) is a valid expression whose type models `sentinel_for<decltype(ranges::rbegin(E))>` with overload resolution performed in a context in which unqualified lookup for `rend` finds only the declarations

```
void rend(auto&) = delete;
void rend(const auto&) = delete;
```

then `ranges::rend(E)` is expression-equivalent to ~~`decay-copy-auto`~~ (`rend(t)`) with overload resolution performed in the above context.
- [...]

Modify 24.3.10 `[range.prim.size]/2` as indicated:

Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:

- If `T` is an array of unknown bound (9.3.4.5), `ranges::size(E)` is ill-formed.
- Otherwise, if `T` is an array type, `ranges::size(E)` is expression-equivalent to ~~`decay-copy_auto`~~ (`extent_v<T>`) .
- Otherwise, if `disable_sized_range<remove_cv_t<T>>` (24.4.3) is false and ~~`decay-copy_auto`~~ (`t.size()`) is a valid expression of integer-like type (23.3.4.4), `ranges::size(E)` is expression-equivalent to ~~`decay-copy_auto`~~ (`t.size()`) .
- Otherwise, if `T` is a class or enumeration type, `disable_sized_range<remove_cv_t<T>>` is false and ~~`decay-copy_auto`~~ (`size(t)`) is a valid expression of integer-like type with overload resolution performed in a context in which unqualified lookup for `size` finds only the declarations

```
void size(auto&) = delete;
void size(const auto&) = delete;
```

then `ranges::size(E)` is expression-equivalent to ~~`decay-copy_auto`~~ (`size(t)`) with overload resolution performed in the above context.
- [...]

Modify 24.3.13 [range.prim.data]/2 as indicated:

Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:

- If `E` is an rvalue and `enable_borrowed_range<remove_cv_t<T>>` is false , `ranges::data(E)` is ill-formed.
- Otherwise, if `T` is an array type (6.8.3) and `remove_all_extents_t<T>` is an incomplete type, `ranges::data(E)` is ill-formed with no diagnostic required.
- Otherwise, if ~~`decay-copy_auto`~~ (`t.data()`) is a valid expression of pointer to object type, `ranges::data(E)` is expression-equivalent to ~~`decay-copy_auto`~~ (`t.data()`) .
- [...]

Modify 24.7.4.1 [range.all.general]/2 as indicated:

The name `views::all` denotes a range adaptor object (24.7.2). Given a subexpression `E`, the expression `views::all(E)` is expression-equivalent to:

- `decay-copy(E)` if the ~~decayed type of `E`~~ type of `auto(E)` models `view` .
- Otherwise, `ref_view{E}` if that expression is well-formed.
- Otherwise, `subrange{E}` .

Modify 32.4.3.3 [thread.thread.constr]/6 as indicated:

Effects: The new thread of execution executes

```
invoke(decay-copyauto(std::forward<F>(f)),  
       decay-copyauto(std::forward<Args>>(args))...)
```

with the calls to ~~decay-copy~~ being evaluated values produced by auto being materialized ([conv.rval] (<http://eel.is/c++draft/conv#rval>)). in the constructing thread. Any return value from this invocation is ignored. [...]

Modify 32.4.4.2 [thread.jthread.cons]/6 as indicated:

Effects: Initializes `ssource` . The new thread of execution executes

```
invoke(decay-copyauto(std::forward<F>(f)), get_stop_token(),  
       decay-copyauto(std::forward<Args>>(args))...)
```

if that expression is well-formed, otherwise

```
invoke(decay-copyauto(std::forward<F>(f)),  
       decay-copyauto(std::forward<Args>>(args))...)
```

with the calls to ~~decay-copy~~ being evaluated values produced by auto being materialized ([conv.rval] (<http://eel.is/c++draft/conv#rval>)). in the constructing thread. Any return value from this invocation is ignored. [...]

Modify 32.9.9 [futures.async]/4 as indicated:

Effects: The first function behaves the same as a call to the second function with a policy argument of `launch::async` | `launch::deferred` [...]:

- If `launch::async` is set in policy, calls `invoke(decay-copyauto(std::forward<F>(f)), decay-copyauto(std::forward<Args>>(args))...)` (20.14.4, 32.4.3.3) as if in a new thread of execution represented by a `thread` object with the calls to ~~decay-copy~~ being evaluated values produced by `auto` being materialized ([`conv.rval`] (<http://eel.is/c++draft/conv#rval>)). in the thread that called `async`. Any return value is stored as the result in the shared state. Any exception propagated from the execution of `invoke(decay-copyauto(std::forward<F>(f)), decay-copyauto(std::forward<Args>>(args))...)` is stored as the exceptional result in the shared state. The `thread` object is stored in the shared state and affects the behavior of any asynchronous return objects that reference that state.
- If `launch::deferred` is set in policy, stores ~~decay-copy~~ `auto` (std::forward<F>(f)) and ~~decay-copy~~ `auto` (std::forward<Args>(args))... in the shared state. These copies of `f` and `args` constitute a deferred function. Invocation of the deferred function evaluates `invoke(std::move(g), std::move(xyz))` where `g` is the stored value of ~~decay-copy~~ `auto` (std::forward<F>(f)) and `xyz` is the stored copy of ~~decay-copy~~ `auto` (std::forward<Args>(args))... . Any return value is stored as the result in the shared state. Any exception propagated from the execution of the deferred function is stored as the exceptional result in the shared state. [...]

Modify 17.11.6 [cmp.alg] as indicated:

[Drafting note: This section proposes a resolution for LWG 3491^[2]. –end note]

The name `strong_order` denotes a customization point object (16.3.3.3.6). Given subexpressions `E` and `F`, the expression `strong_order(E, F)` is expression-equivalent (3.21) to the following:

- ~~If the decayed types of `E` and `F` differ `auto(E)` and `auto(F)` are of different types,~~
`strong_order(E, F)` is ill-formed.
- Otherwise, [...]
- Otherwise, ~~let `T` be `decltype(auto(E))`.~~ ~~if the decayed type `T` of `E`~~ if `T` is a floating-point type, yields a value of type `strong_ordering` that is consistent with the ordering observed by `T`'s comparison operators, and if `numeric_limits<T>::is_iec559` is `true`, is additionally consistent with the `totalOrder` operation as specified in ISO/IEC/IEEE 60559.
- [...]

The name `weak_order` denotes a customization point object (16.3.3.3.6). Given subexpressions `E` and `F`, the expression `weak_order(E, F)` is expression-equivalent (3.21) to the following:

- ~~If the decayed types of `E` and `F` differ `auto(E)` and `auto(F)` are of different types,~~
`weak_order(E, F)` is ill-formed.
- Otherwise, [...]
- Otherwise, ~~let `T` be `decltype(auto(E))`.~~ ~~if the decayed type `T` of `E`~~ if `T` is a floating-point type, yields a value of type `weak_ordering` that is consistent with the ordering observed by `T`'s comparison operators and `strong_order`, and if `numeric_limits<T>::is_iec559` is `true`, [...]

The name `partial_order` denotes a customization point object (16.3.3.3.6). Given subexpressions `E` and `F`, the expression `partial_order(E, F)` is expression-equivalent (3.21) to the following:

- ~~If the decayed types of `E` and `F` differ `auto(E)` and `auto(F)` are of different types,~~
`partial_order(E, F)` is ill-formed.
- [...]

The name `compare_strong_order_fallback` denotes a customization point object (16.3.3.3.6). Given subexpressions `E` and `F`, the expression `compare_strong_order_fallback(E, F)` is expression-equivalent (3.21) to the following:

- ~~If the decayed types of `E` and `F` differ `auto(E)` and `auto(F)` are of different types,~~
`compare_strong_order_fallback(E, F)` is ill-formed.
- [...]

The name `compare_weak_order_fallback` denotes a customization point object (16.3.3.3.6). Given subexpressions `E` and `F`, the expression `compare_weak_order_fallback(E, F)` is expression-equivalent (3.21) to the following:

- ~~If the decayed types of `E` and `F` differ, `auto(E)` and `auto(F)` are of different types,~~
`compare_weak_order_fallback(E, F)` is ill-formed.
- [...]

The name `compare_partial_order_fallback` denotes a customization point object (16.3.3.3.6). Given subexpressions `E` and `F`, the expression `compare_partial_order_fallback(E, F)` is expression-equivalent (3.21) to the following:

- ~~If the decayed types of `E` and `F` differ, `auto(E)` and `auto(F)` are of different types,~~
`compare_partial_order_fallback(E, F)` is ill-formed.
- [...]

Acknowledgments

Thank Alisdair Meredith, Arthur O'Dwyer, and Billy O'Neal for providing examples and feedback for this paper. Thank James Touton for presenting the paper and bringing it forward. Thank Jens Maurer and Casey Carter for reviewing the wording.

References

1. Krügler, Daniel. P0758R0 *Implicit conversion traits and utility functions*. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0758r0.html> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0758r0.html>) ↩
2. Meredith, Alisdair. LWG 3491 *What is a “decayed type?”*
<https://cplusplus.github.io/LWG/issue3491> (<https://cplusplus.github.io/LWG/issue3491>) ↩